

Assignment 4

1. Assignment Description

Problem: Polynomial Addition and Multiplication (50 points)

In this assignment, you are required to implement sparse polynomial operations using C++ linked lists. This system will facilitate the procedural calculation of algebraic expressions, providing functionality for polynomial addition and multiplication without wasting memory on zero-coefficient terms.

Note

Note The initial state of your Linked List must start empty. Because the polynomials can be highly sparse (e.g., $P(x) = 5x^{1000} + 2x^2$), using arrays will result in memory limit exceeded or time limit exceeded. You must use linked lists to strictly store only non-zero terms.

Input

- The first line contains an integer N , representing the number of test cases.
- Each line represents one polynomial. The line starts with an integer T indicating the number of non-zero terms.
- Following T , there are T pairs of numbers separated by spaces. Each pair consists of two integer number representing the `coefficient` and `exponent`. The input pairs are not guaranteed to be sorted by their exponents, and might contain duplicate exponents (which you will need to combine).

Constraints

- number of test cases N : $1 \leq N \leq 20$.
- Number of terms T : $1 \leq T \leq 100$.
- `coefficient`: $-2^{31} \leq \text{coefficient} \leq 2^{31}-1$.
- `exponent`: $-2^{31} \leq \text{exponent} \leq 2^{31}-1$.

Output

- For each test case, simulate the polynomial addition and multiplication, and output the results to the terminal.
- Each output must contain two lines per test case. The first line must start with the prefix "Add: ", and the second line must start with the prefix "Mul: ".
- Following the prefix, output the pairs of `coefficient` and `exponent` separated by spaces.
- The output terms must be sorted in descending order by their `exponent` (from largest to smallest). You must not output any terms with a `coefficient` of 0. If the resulting polynomial is completely empty (all terms cancel out to zero), simply output "0 0"($0x^0$).

Sample Input

$$A = 10x^{1000} + 5x^{50} + x^1$$

$$B = 3x^{500} + 2x^{50}$$

```
1
3 10 1000 5 50 1 1
2 3 500 2 50
```

Sample Output

$$A + B = 10x^{1000} + 3x^{500} + 7x^{50} + x^1$$

$$A \times B = 30x^{1500} + 20x^{1050} + 15x^{550} + 3x^{501} + 10x^{100} + 2x^{51}$$

Add: 10 1000 3 500 7 50 1 1

Mul: 30 1500 20 1050 15 550 3 501 10 100 2 51

2. Submission Guidelines

Please organize your code and files according to the following directory structure and compress them into a single .zip archive before uploading.

- **Single Submission Archive (.zip):** Place your main function code (main.cpp), all custom .cpp and .h files, and any other files required for compilation into a specific root folder. Compress this folder into a .zip file. (Note: If you organize your source codes into multiple subdirectories, you must include a Makefile in the root of your folder.)
- **The build Directory:** You must create a subfolder named build inside your root directory. This folder must contain your compiled executable file (.exe). Any required dependencies (e.g., .dll files).

Naming Example & Directory Structure:

- **Zip File Name:** 1142DS_<Student ID>_a4.zip
(e.g., 1142DS_B112345678_a4.zip)

Your .zip archive must follow this file structure:

```
1142DS_B123456789_a4.zip/
├── build/ # Must contain executable
│   ├── 1142DS_B123456789_a4.exe
│   └── ... # dependencies (.dll)
├── main.cpp
└── ... # And any other .cpp/.h/Makefile files
```

3. Grading Policies

General Criteria (15 points):

- **Coding Style (10 points):** Assessed based on code readability, consistent indentation, and meaningful naming conventions. Arbitrary or meaningless variable names (e.g., a, b, c, aa, bb, cc) are strictly prohibited and will result in point deductions.
- **Comments (5 points):** Assessed based on brief and clear explanations of your function purposes and the overall processing flow. Focus on explaining the core logic. Do not add trivial or redundant comments for basic C++ syntax (e.g., commenting on #include or using namespace std). Additionally, every source file must include a file header specifying the author (including student ID), the date of writing (e.g., Mar. 22, 2026), and a description of the program's functionality and usage.

Problem-Specific Criteria (35 points)

- **Correctness (30 points):** Points are awarded based on the percentage of test cases your program successfully passes.
- **Bonus (5 points):** Instead of a basic struct setup, implement a cohesive Linked-list class and utilize C++ template (e.g., template <typename T> class

`LinkedList`). This allows your linked list data structure to be easily reused for other data types in future applications.

4. Restrictions (Not Allowed)

Unless explicitly permitted by the assignment prompt, the usage of container libraries in the C++ Standard Template Library (STL)—such as `vector`, `list`, `map`, `set`, `deque`, or similar container structures—is strictly prohibited. If such libraries are used, the correctness score will be automatically **marked as 0**.